EX 8: IMAGE GENERATION USING VARIATIONAL AUTOENCODER (VAE)

AIM

To implement a Variational Autoencoder (VAE) using PyTorch and train it on the CelebA dataset to learn latent representations and generate new synthetic face images.

PROCEDURE

1. **Import the required libraries, including PyTorch, torchvision, and Matplotlib.**
2. **Load and preprocess the CelebA dataset by resizing and normalizing the images.**
3. **Define the encoder to extract latent distribution parameters, mean and log variance.**
4. **Define the decoder to reconstruct images from latent vectors.**
5. **Implement the reparameterization trick to sample latent vectors.**
6. **Define the VAE loss function using reconstruction loss and KL divergence.**
7. **Train the model using the Adam optimizer.**
8. **Generate images by sampling random latent vectors and visualize the results.**

CODE

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, Subset
import matplotlib.pyplot as plt


# Configure parameters
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
image_size = 64
batch_size = 64
latent_dim = 100
num_epochs = 5
learning_rate = 1e-3


# Load and preprocess CelebA dataset
transform = transforms.Compose([
    transforms.CenterCrop(178),
```

```
    transforms.Resize((image_size, image_size)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

dataset = datasets.CelebA(
    root="data",
    split="train",
    download=True,
    transform=transform
)

# Use a subset for faster training
dataset = Subset(dataset, range(500))
dataloader = DataLoader(
    dataset,
    batch_size=batch_size,
    shuffle=True
)



# Define Encoder
class Encoder(nn.Module):
    def __init__(self, latent_dim):
        super().__init__()

        self.conv = nn.Sequential(
            nn.Conv2d(3, 64, 4, 2, 1),
            nn.ReLU(),
            nn.Conv2d(64, 128, 4, 2, 1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.Conv2d(128, 256, 4, 2, 1),
```

```
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.Conv2d(256, 512, 4, 2, 1),
            nn.BatchNorm2d(512),
            nn.ReLU()
        )

        self.fc_mu = nn.Linear(512 * 4 * 4, latent_dim)
        self.fc_logvar = nn.Linear(512 * 4 * 4, latent_dim)

    def forward(self, x):
        x = self.conv(x)
        x = x.view(x.size(0), -1)
        return self.fc_mu(x), self.fc_logvar(x)


# Define Decoder
class Decoder(nn.Module):
    def __init__(self, latent_dim):
        super().__init__()

        self.fc = nn.Linear(latent_dim, 512 * 4 * 4)

        self.deconv = nn.Sequential(
            nn.ConvTranspose2d(512, 256, 4, 2, 1),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.ConvTranspose2d(256, 128, 4, 2, 1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.ConvTranspose2d(128, 64, 4, 2, 1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
```

```
            nn.ConvTranspose2d(64, 3, 4, 2, 1),
            nn.Tanh()
        )

    def forward(self, z):
        x = self.fc(z)
        x = x.view(x.size(0), 512, 4, 4)
        return self.deconv(x)


# Define Variational Autoencoder
class VAE(nn.Module):
    def __init__(self, latent_dim):
        super().__init__()
        self.encoder = Encoder(latent_dim)
        self.decoder = Decoder(latent_dim)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def forward(self, x):
        mu, logvar = self.encoder(x)
        z = self.reparameterize(mu, logvar)
        return self.decoder(z), mu, logvar


# Define VAE loss function
def vae_loss(recon_x, x, mu, logvar):
    recon_loss = F.mse_loss(recon_x, x, reduction="sum")
    kl_div = -0.5 * torch.sum(
        1 + logvar - mu.pow(2) - logvar.exp()
```

```
    )
    return recon_loss + kl_div


# Train the model
model = VAE(latent_dim).to(device)
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=learning_rate
)

for epoch in range(num_epochs):
    model.train()
    total_loss = 0

    for images, _ in dataloader:
        images = images.to(device)

        recon, mu, logvar = model(images)
        loss = vae_loss(recon, images, mu, logvar)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(dataloader.dataset)
    print(f"Epoch [{epoch + 1}/{num_epochs}], Loss: {avg_loss:.4f}")

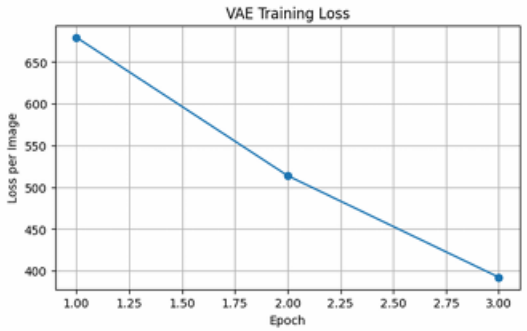
    # Generate new images
    model.eval()
    with torch.no_grad():
```

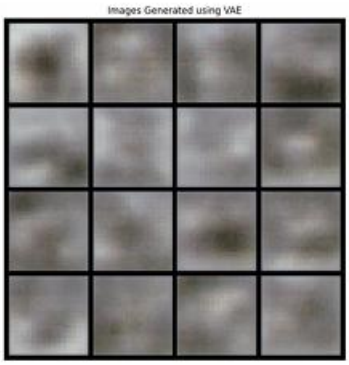
```
    z = torch.randn(64, latent_dim).to(device)
    sample_images = model.decoder(z).cpu()
    sample_images = sample_images * 0.5 + 0.5

    grid = torchvision.utils.make_grid(
      sample_images.clamp(0, 1),
      nrow=8
    )

    plt.figure(figsize=(8, 8))
    plt.imshow(grid.permute(1, 2, 0).numpy())
    plt.axis("off")
    plt.title(f"Generated Faces - Epoch {epoch + 1}")
    plt.show()
```

OUTPUT





RESULT

Thus, a Variational Autoencoder was implemented using PyTorch and trained on a subset of the CelebA dataset. The model learned latent representations of facial images and generated synthetic images by sampling from the latent space.